

Enterprise AI Platform - Etapas 11 a 15

Extension avanzada del roadmap de AI Engineering 100% Python

Estas cinco etapas se realizan **despues de completar las etapas 1-10**. Su objetivo es llevar el proyecto desde una plataforma AI completa a una implementacion con mayor profundidad en ML Engineering, serving, sistemas distribuidos, costos y experimentacion.

Etapas	Bloque	Objetivo
11	Advanced ML Engineering	Lifecycle ML completo, drift y retraining
12	Advanced Model Serving & Inference	Serving eficiente de modelos y LLMs
13	AI Platform & Distributed AI	Arquitectura distribuida y model routing
14	AI FinOps & Optimization	Optimizar costo, latencia y calidad
15	Advanced Data & Experimentation	Datos, experimentos y releases AI reproducibles

ETAPA 11 - Advanced ML Engineering

Completar el ciclo de vida de Machine Learning mas alla de entrenar y registrar un modelo.

Que implementar

- Feature engineering avanzado y pipelines reproducibles.
- Feature Store: reutilizacion y consistencia de features entre training e inference.
- Data drift, feature drift y concept drift.
- Deteccion de degradacion del modelo.
- Retraining automatico basado en condiciones y metricas.
- Champion/challenger para comparar modelos en produccion.
- Versionado y rollback de modelos.
- Validacion de datos y modelos antes del despliegue.

Stack recomendado

Python, scikit-learn/PyTorch, MLflow, herramientas de validacion de datos y un Feature Store cuando aporte valor.

Criterio de finalizacion

El sistema detecta cambios relevantes, registra la degradacion y puede ejecutar de forma controlada un pipeline de retraining, evaluacion y promocion de modelo.

ETAPA 12 - Advanced Model Serving & Inference

Aprender a servir modelos propios y LLMs de forma eficiente, midiendo capacidad y uso de recursos.

Que implementar

- Serving de modelos open-source.
- vLLM o motor equivalente para inferencia LLM.
- Batching y continuous batching.

- KV cache y fundamentos de memoria durante inferencia.
- Quantization y precision reducida.
- Warmup y carga de modelos.
- Throughput, requests/second y latencia p50/p95/p99.
- Uso de CPU vs GPU y fundamentos de memoria GPU.
- Streaming de tokens.
- Benchmarks reproducibles de inferencia.

Stack recomendado

Python, FastAPI, PyTorch, Hugging Face, vLLM, Docker y metricas de serving.

Criterio de finalizacion

Existe un servicio de inferencia medible que puede comparar configuraciones/modelos por calidad, latencia, throughput y consumo.

ETAPA 13 - AI Platform & Distributed AI

Evolucionar la aplicacion hacia una plataforma con multiples modelos, servicios, workers y rutas de ejecucion.

Que implementar

- AI/Model Gateway.
- Model routing segun tarea, calidad, costo o latencia.
- Multiples proveedores y modelos locales.
- Fallback automatico cuando un proveedor/modelo falla.
- Queues y workers para tareas pesadas.
- Async inference y procesamiento en background.
- Load balancing y autoscaling.
- Servicios separados para RAG, agentes, ML e inference cuando sea necesario.
- Idempotencia y manejo de tareas repetidas.
- Health checks, readiness y degradacion controlada.

Stack recomendado

Python, FastAPI, Redis/queue system, workers, Docker; cloud/autoscaling en la version desplegada.

Criterio de finalizacion

Una request puede ser enrutada al recurso correcto, las tareas pesadas se procesan asincronicamente y el sistema tolera fallos de componentes.

ETAPA 14 - AI FinOps & Optimization

Medir y optimizar el costo real de operar el sistema sin sacrificar innecesariamente calidad o rendimiento.

Que implementar

- Costo por request, usuario, feature y modelo.
- Tracking de tokens de entrada/salida.
- Presupuestos y limites de consumo.
- Model routing entre modelos baratos y modelos potentes.
- Prompt/context optimization para reducir tokens.
- Response caching y semantic caching.
- Control de concurrencia y rate limits.
- Medicion de costo de infraestructura/GPU cuando corresponda.
- Dashboards de costo y alertas.
- Evaluar el trade-off Quality vs Latency vs Cost.

Stack recomendado

Python, metricas de proveedores/modelos, cache, observabilidad y dashboards.

Criterio de finalizacion

El proyecto puede explicar cuanto cuesta una operacion AI y seleccionar estrategias/modelos utilizando metricas de calidad, latencia y costo.

ETAPA 15 - Advanced Data & Experimentation

Cerrar el roadmap con una capa madura de datos, experimentacion y despliegues controlados.

Que implementar

- Dataset versioning y data lineage.
- Validacion y calidad de datasets.
- Synthetic data con controles de calidad.
- Experiment tracking reproducible.
- Reproducibilidad de training/evals.
- A/B testing.
- Champion/challenger experiments.
- Shadow deployment.
- Canary releases para modelos/prompts.
- Offline vs online evaluation.
- Comparacion sistematica de prompts, modelos y pipelines.
- Registro de artefactos, configuraciones, metricas y resultados.

Stack recomendado

Python, MLflow, dataset/versioning tooling, pipelines de evaluacion y CI/CD.

Criterio de finalizacion

Cada cambio importante en modelo, prompt, retrieval o pipeline puede compararse contra una baseline y desplegarse de forma controlada.

Flujo final despues de las 15 etapas

Datos -> validacion -> features/datasets -> training/fine-tuning -> experiment tracking.

Evaluacion -> Model Registry -> serving -> routing -> usuarios/agentes.

Observabilidad -> drift/calidad/costo -> decisiones de retraining o rollback.

CI/CD + releases controlados -> shadow/canary -> produccion.

Resultado esperado para portfolio

Al finalizar las etapas 1-15, Enterprise AI Platform debe demostrar AI Engineering end-to-end en Python: LLMs, RAG, agentes, MCP, ML/DL, Transformers, fine-tuning, multimodal, MLOps/LLMOps, evaluacion, observabilidad, seguridad, ML lifecycle, model serving, sistemas distribuidos, FinOps, data engineering y experimentacion controlada.

Recomendacion: publicar una V1 al terminar las etapas 1-10 y tratar 11-15 como una V2 avanzada. No es necesario esperar a completar estas cinco etapas para empezar a mostrar el proyecto.